UNIVERSITÉ Sultan Moulay Slimane

Faculté des Sciences / Département d'Informatique

Filière / Master : AIDC



TP 2 : Simulation Multi-Agents avec Mesa

Modélisation de la diffusion d'opinions et dynamique de groupe

Réalisé par :

AHLAME LAOUAD

Encadré par :

Pr. Lhoussaine Daoudi

Année Universitaire : 2026-2027

Date : Octobre 2026

1 Introduction

Ce travail pratique est consacré à la modélisation et à l'expérimentation de systèmes multi-agents (SMA) à l'aide de la bibliothèque Python **Mesa**. En essayant de faire la modélisation d'un système d'opinions pour une population et le test de l'influence des opinions des voisins sur le changement d'avis de l'individu choisi.

2 Objectifs du TP

— Implémenter les agents autonomes et définir leurs règles d'interaction locale.
— Modéliser la grille d'espace et les variables d'état (opinion, influence α).
— Analyser l'émergence globale du consensus à l'aide des visualisations.
— test de l'influence du changement de la valeur alpha sur la convergence des avis.

3 Protocole ODD(Overview, Design, Details :

ODD est un standard international utilisé pour décrire et documenter de manière claire, rigoureuse et reproductible les modèles d'inspiration individuelle ou multi-agents . Il se base sur 3 concepts globales :

— **Overview :**Fournit une vue globale sur le système et sa structure.Partant de l'objectif , des entités , l'environnement passant à l'ordre dans lequel les actions des agents s'exécutent au cours d'une étape de simulation.
— **Design :**Explique les principes théoriques et conceptuels qui guident le comportement des agents.Inclut des notions clés comme l'émergence, l'adaptation, l'apprentissage, l'objectif , la prédiction, la perception, l'interaction et la stochasticité.
— **Details :**Fournit les informations techniques et les sous-modèles nécessaires pour recréer le code.Notament : l'initialisation ,les données d'entrée et les sous modèles.

4 Installation des tecnologies et bibliothèques nécessaire :

Pour tester notre simulation , on a met en place un environnement :

— **Language de programmation :**Python 3.14.6
— **Bibliothèques :** pour une execution cohérente on a installé les bibliothèques suivantes :

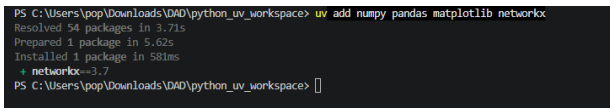


FIGURE 1 – Installation des bibliothèques python

— **Framework :** Pour simuler notre application multi agent on a utilisé mesa come framwork

5 L'implémentation de l'agent :

Pour implémenter la simulation, on a créer ces différentes classes :

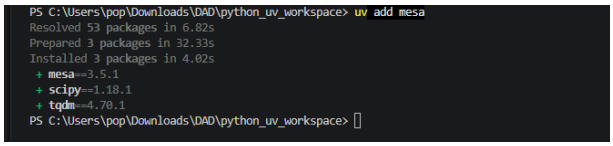


FIGURE 2 – Installation mesa

1. **CitizenAgent :**Cette classe à pour but de récupérer la récupérer la liste de ses voisins sur la grille et calcule la moyenne de leurs opinions en se basant sur la variable d'état "opinion". Alors que l'individu concerné rapproche son propre opinion de la moyenne proportionnelle en calibrant le coefficient d'influence α .

Implémentation de l'agent : `CitizenAgent`

```python
class CitizenAgent(mesa.Agent):
    """Un individu ayant une opinion qui evolue selon ses voisins."""
    def __init__(self, model, alpha):
        # Initialisation de l'agent (enregistrement automatique dans Mesa 3)
        super().__init__(model)

        # Opinion initiale tiree aleatoirement entre 0 et 1
        self.opinion = self.model.random.random()
        # Niveau d'influence sociale
        self.alpha = alpha

    def step(self):
        # Recuperation des voisins (Moore = 8 voisins alentours, sans le centre)
        neighbors = self.model.grid.get_neighbors(self.pos, moore=True, include_center=False)

        if len(neighbors) > 0:
            # Calcul de l'opinion moyenne des voisins
            mean_opinion = np.mean([neighbor.opinion for neighbor in neighbors])

            # Mise a jour de l'opinion selon la formule mathematique du TD
            self.opinion = self.opinion + self.alpha * (mean_opinion - self.opinion)
```

2. **OpinionModel(population) :**initialise l'espace de grille autrement l'environnement d'agents, elle gére le temps ainsi la collecte des données.

Implémentation de la grille : `OpinionModel`

```python
class OpinionModel(mesa.Model):
    """Modèle simulant la formation de l'opinion publique sur une grille."""
    def __init__(self, width, height, alpha=0.2, seed=None):
        super().__init__(seed=seed)

        # Espace discret (grille) avec bords communicants (torus=True)
        self.grid = mesa.space.MultiGrid(width, height, torus=True)
        self.alpha = alpha

        # Initialisation de la population : on place un agent par case
        for _, (x, y) in self.grid.coord_iter():
            agent = CitizenAgent(self, self.alpha)
            self.grid.place_agent(agent, (x, y))

        # Mise en place du collecteur de données pour l'observation
        self.datacollector = mesa.DataCollector(
            model_reporters={
                "Opinion Moyenne": lambda m: np.mean([a.opinion for a in m.agents]),
                "Variance (Dispersion)": lambda m: np.var([a.opinion for a in m.agents])
            },
            agent_reporters={
                "Opinion": "opinion"
            }
        )

    def step(self):
        # 1. Collecter les données de l'état actuel
        self.datacollector.collect(self)
        # 2. Activer tous les agents dans un ordre aléatoire (Syntaxe Mesa 3)
        self.agents.shuffle_do("step")
```

6 L'éxecution de la simulation et observation :

Dans cette partie nous allons instancier notre modèle pour tester l'évolution globale et l'évolution individuelle pour identifier l'émergence d'un consensus.

6.1 Code éxecuté :

Implémentation de l'agent : `CitizenAgent`

```
# Paramètres du modèle
WIDTH = 10
HEIGHT = 10
ALPHA = 0.5  # Modifiez alpha pour observer son impact sur la vitesse de convergence
STEPS = 50

# Création et exécution du modèle
model = OpinionModel(width=WIDTH, height=HEIGHT, alpha=ALPHA, seed=42)

for i in range(STEPS):
    model.step()

# Récupération des données collectées
model_data = model.datacollector.get_model_vars_dataframe()
agent_data = model.datacollector.get_agent_vars_dataframe()

# Affichage des résultats globaux (Moyenne et Variance)
fig, ax1 = plt.subplots(figsize=(10, 5))

color = 'tab:blue'
ax1.set_xlabel('Ticks (Temps)')
ax1.set_ylabel('Opinion Moyenne', color=color)
ax1.plot(model_data.index, model_data['Opinion Moyenne'], color=color, label="Moyenne")
ax1.tick_params(axis='y', labelcolor=color)
ax1.set_ylim(0, 1)

ax2 = ax1.twinx()
color = 'tab:red'
ax2.set_ylabel('Variance (Dispersion)', color=color)
ax2.plot(model_data.index, model_data['Variance (Dispersion)'], color=color, linestyle='--', label="Variance")
ax2.tick_params(axis='y', labelcolor=color)

plt.title("Évolution globale de l'opinion (Moyenne et Dispersion)")
fig.tight_layout()
plt.show()

# Affichage des trajectoires individuelles (Polarisation vs Consensus)
# On prend un échantillon de 20 agents pour la lisibilité
agent_opinions = agent_data.unstack(level=1)['Opinion']
sample_agents = agent_opinions.columns[:20]

```

```
plt.figure(figsize=(10, 5))
for agent_id in sample_agents:
    plt.plot(agent_opinions.index, agent_opinions[agent_id],
       alpha=0.6)

plt.title("Trajectoires des opinions individuelles (É
   chantillon de 20 agents)")
plt.xlabel("Ticks (Temps)")
plt.ylabel("Opinion [0, 1]")
plt.ylim(0, 1)
plt.show()
```

6.2 Résultat :

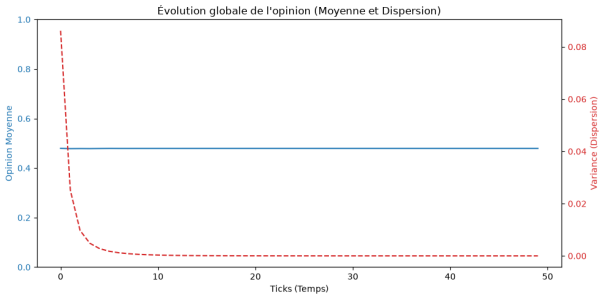


FIGURE 3 – Evolution globale de l'opinion

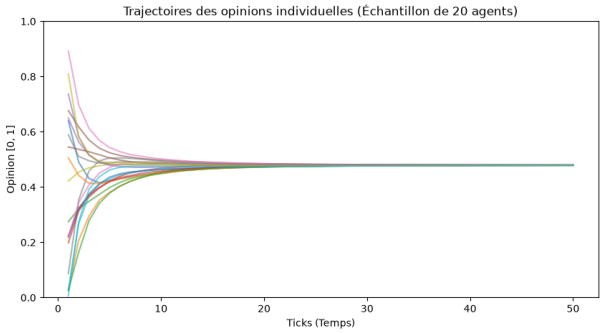


FIGURE 4 – Trajectoires des opinions des individus pour $\alpha = 0.5$

7 Réponses aux questions de compréhension :

7.1 Compréhension théorique :

7.1.1 Q1_Impact de α :

Lors du test des différentes valeurs α, on a constaté que lorsque α est entre 0 et 1, l'opinion de l'individu s'approche aux opinions de la population à chaque pas (step/ interaction). Pour bien comprendre voilà un exmple visuel pour voir la différence :

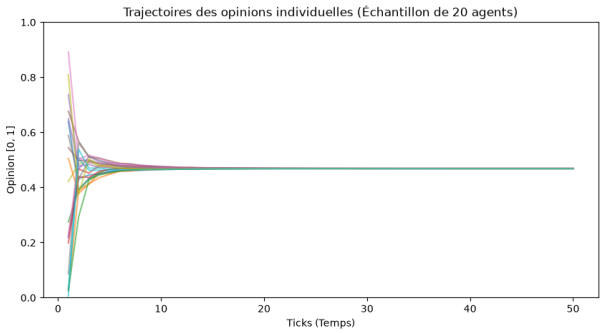


FIGURE 5 – Cas de $\alpha = 1$

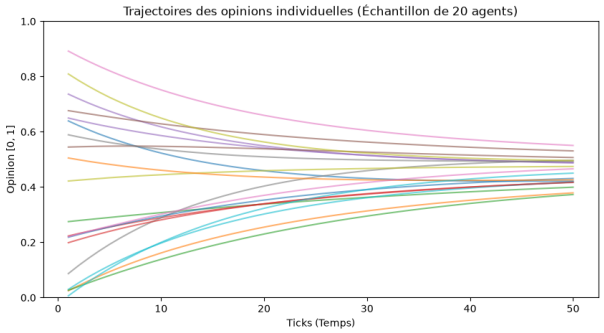


FIGURE 6 – Cas de α =0.05

7.1.2 Q2_Le mécanisme mathématique de convergence :

La méthode "step() " utilise la méthode de la moyenne pondéré pour qu'à chaque pas fait la mise à jour de l'opinion de l'individu, et presque la valeur reste toujours entre 0 et 1 donc la convexité assure que l'avis d elindividu ne sort jamais des avis de l'intervalle définit par les opinions utilisés pour faire la mise à jour. Et pour qu'on reste dans le consensus on doit faire une mise à jour dans la direction de minimiser la différence entre l'avis et l'avis du groupe.

7.1.3 Q3_Notion d'emergence dans ce cas :

La notion d'emergenece se manifeste dans notre exemple dans l'absence d'un agent superviseur qui oriente le consensus, dont chaque agent à son propre avis et à chque step l'avis peronel s'aligne avec un autre et vers la fin en peut avoir un consensus d'avis l'un est proche de l'autre. Donc le consensus est un phénomène emergent dans ce cas.

7.2 Techniques de manipulation de mesa :

7.2.1 Q1_ Différence entre activatin des agents dans les syntaxes : shuffle_do et do :

Pour l'activation des agents gérée dans la nouvelle syntaxe de Mesa 3 (‘self.agents.shuffle_do("step")‘) sert à appliquer l'itération de moyenne pondéré pour mise à jour de l'avis en changeant l'ordre des agents du point de départ, pour but d'éviter toute type de biais dans le chanegment d'avis qui doit être normal sans être biaisé ou forcé à être choisi. Par cotre la méthode do simple garde l'ordre des agents tel qu'il est et cela peut être un biais qui apparait lors du calcul des avis.

7.2.2 Q2_la différence entre ‘model_reporters‘ et ‘agent_reporters‘ :

Pour model_reporters on collecte les données de toute la population, ainsi la descriptin d’état globale de la grille. Pour agent_reporters on collecte des données de chaque agent seul. Dans les graphes qu’on a model_repoters nous informe du consensus alors la dispersion. Donc La moyenne des opinions permet de suivre l’évolution globale de la population, tandis que la variance mesure leur dispersion ; les deux indicateurs, complétés par les opinions individuelles, permettent d’évaluer la convergence vers un consensus et de détecter une éventuelle polarisation.

7.2.3 Q3_La récupération des voisins au cas de x=0 :

Lorsque le paramètre torus=True est activé, la grille se comporte comme un espace sphérique en 3D (un tore) où les bords opposés sont connectés de manière continue. Si un agent se trouve à l’extrême limite x=0, le système ne rencontre aucun "trou" ou mur invisible : la récupération des voisins se fait tout à fait normalement en "bouclant" de l’autre côté de la grille (vers x=taille_max). Cela permet aux agents de naviguer sans interruption, tout en exigeant une vigilance accrue dans le code car deux agents situés aux deux extrémités géométriques de la carte se retrouveront virtuellement collés l’un à l’autre et interagiront comme des voisins directs.

7.3 Ouverture et esprit critique :

7.3.1 Q1_Modification code pour introduire biais de confirmation :

Pour implémenter ce phénomène, on doit introduire le changement dans la fonction step en changeant la méthode de moyenne pondérée, par ce code :

Implémentation de l’agent : `CitizenAgent`

```
def step(self):
    neighbors = self.model.grid.get_neighbors(
        self.pos,
        moore=True,
        include_center=False,
        radius=1
    )

    if neighbors:
        neighbor = self.random.choice(neighbors)

        # Biais de confirmation : vérifier la proximité des
           opinions
        if abs(self.opinion - neighbor.opinion) < self.
           epsilon:
            self.opinion += self.alpha * (
                neighbor.opinion - self.opinion
            )
```

7.3.2 Q2_ Modélisation d'un réseau social :

our modéliser un réseau social "network" en peut utiliser la bibliothèque NetworkGrid dont on va simuler un graphe à la place d'une grille. Dans ce cas contrairement à une grille spatiale, NetworkGrid permet de représenter des relations sociales irrégulières et des différences de connectivité, comme dans un réseau d'abonnés.

8 Conclusion :

Dans cet atelier nu avons pu traiter le cas de polarisation des opinions et le suivi d'un consensus, en utilisant la méthode de la moyenne pondérée pour une mise à jour d'opinion à chaque step, de plus nous avons découvert les différentes propriété de la bibliothèque mesa qui nous a permet de mettre en place des connaissance de grille, réseau, collecte données, émergnece ....

Donc maintenant on a une base solide pour les futur atelier de création et simulation des cas d'études des systèmes agentiques.